

Secure File Encryption Tool

Complete Code Explanation

1. Project Overview

This repository is a local Python desktop application for encrypting and decrypting files with a password. The GUI is built with Tkinter. The cryptographic core uses AES-256-GCM, which provides both confidentiality and authentication. Passwords are converted into encryption keys with PBKDF2-HMAC-SHA256 or Argon2id.

The application creates files in the SFET format. Each encrypted file contains a version, KDF identifier, KDF parameter, salt, nonce, original filename, original file size, ciphertext, and the AES-GCM authentication tag. Passwords and derived keys are never stored in the file.

2. Application Flow

1. main.py starts the application.
2. gui.py forwards startup to ui/main_window.py.
3. The user selects a file, enters a password, and chooses a KDF.
4. crypto_utils.py validates the input, derives a key, encrypts or decrypts the file, and returns a result dictionary.
5. features/safe_io.py writes successful output through a temporary file and atomic replacement.
6. The GUI displays the result and records safe activity in the log.

3. Root-Level Files

``main.py``

This is the application entry point. Its module docstring explains its purpose. It imports main from gui.py. The if `__name__ == "__main__"` guard ensures the GUI starts only when the file is run directly. Calling `main()` creates the application and starts Tkinter's event loop.

``gui.py``

This is a compatibility wrapper. It imports `SecureFileApp` from `ui.main_window`. Its `main()` function creates the window and calls `mainloop()`. Keeping this wrapper means existing commands such as `python gui.py continue` to work while the GUI remains modular.

``crypto_utils.py``

This is the security core. It imports operating-system randomness, binary packing, filesystem paths, AES-GCM, and the reusable KDF and safe-I/O helpers.

The constants define the SFET magic value, file-format version, nonce size, maximum filename size, file-size field size, and minimum header size. The custom exception classes describe malformed headers and authentication failures.

`is_argon2_available`, `write_atomic_file`, and `get_kdf_name` are compatibility wrappers around the feature modules. They keep the public API simple while the implementation is split into focused files.

`build_aad` creates the Associated Authenticated Data. It validates the filename and file size, then packs the version, KDF ID, filename length, filename, and file size into deterministic bytes. AES-GCM authenticates this metadata even though it is not encrypted.

`create_sfet_header` validates all header fields and serializes them in this order: magic, version, KDF ID, KDF parameter, salt, nonce, filename length, file size, and filename.

`parse_sfet_header` performs the reverse operation. It checks the magic value, version, KDF ID, parameter, salt, nonce, filename boundaries, and minimum ciphertext length. It returns a dictionary containing the parsed metadata and ciphertext.

`derive_key` forwards password processing to `features/kdf_utils.py`.

`encrypt_file` validates the input path, reads the plaintext, encodes the original filename, selects PBKDF2 or Argon2id, generates a random salt and nonce, derives a 256-bit key, builds AAD, encrypts with AES-GCM, creates the SFET header, refuses to overwrite an existing output, and atomically writes the header plus ciphertext. It catches common file, validation, and cryptographic errors and returns a dictionary containing success, message, and sometimes `output_path`.

`decrypt_file` validates the encrypted input, parses its header, derives the key using the stored KDF information, reconstructs AAD, and calls AES-GCM decryption. An `InvalidTag` exception means the password, ciphertext, or authenticated metadata is wrong, so no plaintext is written. It also checks the plaintext length, refuses to overwrite existing output, writes atomically, and returns a result dictionary.

The `__all__` list identifies the public constants, helper functions, exception classes, and encryption functions.

`file\_utils.py`

This module contains general file operations. `read_file_in_chunks` delegates to the safe-I/O implementation. `calculate_sha256` validates the path, creates a SHA-256 object, reads the file in chunks, updates the digest, and returns the hexadecimal digest.

`validate_password_strength` delegates to the password-validation feature. `ensure_parent_directory` creates missing parent folders. `resolve_output_path` chooses a sensible encrypted or decrypted filename based on the input path and user selection.

`setup_logging` creates the logs directory, configures an INFO logger, and adds a file handler only once. `log_activity` obtains the logger and records a message without storing passwords or encryption keys.

`requirements.txt`

This lists the external dependencies required by the project, including cryptography for AES-GCM and argon2-cffi for optional Argon2id support.

4. Feature Modules

`features/kdf\_utils.py`

This module implements password-based key derivation. It attempts to import Argon2id and records whether the optional dependency is available. Constants identify PBKDF2 and Argon2id, set PBKDF2's 600,000 iterations, set

Argon2id's time cost, define the 16-byte salt, and define the 32-byte AES key.

`is_argon2_available` reports dependency availability. `get_kdf_name` converts a numeric KDF ID into a readable label. `derive_key` validates the password and salt, then uses PBKDF2-HMAC-SHA256 or Argon2id to return exactly 32 bytes.

`features/password\_validation.py`

`validate_password_strength` checks password type, minimum length, uppercase letters, lowercase letters, digits, and special characters. It counts passed checks, labels the result Weak, Medium, or Strong, and returns the score, label, and individual criteria.

`features/safe\_io.py`

`read_file_in_chunks` opens a file in binary mode and yields 64 KB blocks. `write_atomic_file` creates a temporary sibling file, writes the complete content, then calls `os.replace` so the final path changes only after a successful write. `ensure_parent_directory` creates the destination folder when needed.

`features/batch\_operations.py`

`encrypt_files` loops over a collection of input paths, chooses an output directory or next-to-input destination, calls `encrypt_file`, and returns successful output paths. `decrypt_files` follows the same pattern and removes `.enc` from output names.

`features/\_\_init\_\_.py`

This package initializer exposes feature functions through lazy imports. `__getattr__` imports a requested function only when it is accessed. This avoids eager imports and prevents circular dependencies between the feature package and the cryptographic core.

5. User Interface Modules

`ui/main\_window.py`

`SecureFileApp` subclasses `tk.Tk` and represents the main window. The constructor sets the title and size, creates Tkinter variables for the selected path, output path, password, status, SHA-256 result, file size, current task, and KDF choice, then builds the interface.

`_build_ui` creates the file picker, password entry, show-password button, KDF menu, output picker, encrypt/decrypt buttons, SHA-256 button, password-strength label, hash display, and status display. The password entry binds key-release events to the strength indicator.

`_set_busy` disables the main input controls during a background operation. `_update_file_size` asks the helper module for a readable size label. `browse_input_file` and `browse_output_dir` use Tkinter dialogs to select paths.

`toggle_password_visibility` changes the password entry between masked and visible text. `update_strength_indicator` calls the validation helper and changes the label text and color.

`calculate_sha_button` validates that a file exists, calculates its SHA-256 digest, updates the display, and records the activity. `_resolve_kdf_id` converts the selected menu label into its numeric identifier.

`_resolve_output_for_operation` chooses encrypted and decrypted output names. For decryption it reads the original filename from the SFET header when possible. `_validate_inputs` checks the file, password length, and optional Argon2id availability.

`_run_operation` validates inputs, asks for overwrite confirmation, marks the UI busy, and starts a daemon worker thread. `_worker` calls the cryptographic function outside the GUI thread, then schedules `_handle_result` on Tkinter's main thread with `after`.

`_handle_result` restores the UI, updates the status, records activity, displays errors or success dialogs, and displays hash information after decryption. `encrypt_file` and `decrypt_file` are button callbacks that start the corresponding operation.

``ui/helpers.py``

`format_file_size` converts a byte count into B, KB, MB, or GB. `get_selected_file_size` reads a selected file's size and returns a user-facing label, handling missing or unreadable paths safely.

``ui/__init__.py``

This marks `ui` as a Python package and exposes `SecureFileApp` as its public application class.

6. Tests

``tests/test_crypto.py``

The tests use `unittest` and temporary directories so they test real filesystem and cryptographic behavior without modifying project data.

`test_round_trip_and_randomized_ciphertext` encrypts and decrypts a file, verifies the recovered bytes, and confirms that encrypting the same plaintext twice produces different encrypted files because salt and nonce values are randomized.

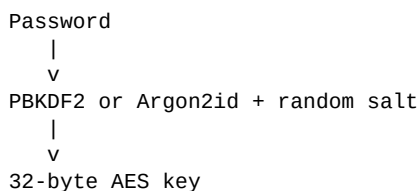
`test_wrong_password_and_tampering` verifies that an incorrect password and modified ciphertext both fail AES-GCM authentication.

`test_atomic_file_write_and_argon2id_key_derivation` checks that Argon2id creates a 32-byte key and that atomic writes create the target without leaving the temporary file behind.

`test_header_validation_and_aad_rejection` checks invalid headers, parsed metadata, salt and nonce sizes, and authentication failure after modifying authenticated header metadata.

`test_empty_file_round_trip` proves that zero-byte files are supported. `test_output_file_conflict_is_rejected` proves that existing encrypted and decrypted targets are not overwritten.

7. Security Flow



|
v
AES-256-GCM + random nonce + authenticated metadata
|
v
SFET header + ciphertext + authentication tag

During decryption, AES-GCM must verify the authentication tag before any output is written. Therefore, a wrong password, modified file, damaged header, or corrupted ciphertext causes the operation to fail safely.

8. Important Implementation Note

The cryptographic integrity check is AES-GCM authentication. The GUI's post-decryption SHA-256 display currently hashes the encrypted input file and compares that value with the decrypted output file. Those are different files, so that display can report FAILED even after valid decryption. The encryption and decryption authentication logic remains the authoritative integrity check.

9. Running the Project

Install dependencies with `pip install -r requirements.txt`, start the GUI with `python main.py`, and run tests with `python -m unittest discover -s tests -v`.